

Guide d'architecture

Fad Agency OS

Pour qui ? Pour le propriétaire de la plateforme, qui n'est pas développeur.

Pour quoi ? Comprendre comment le logiciel est construit, et savoir **où aller** quand on veut modifier quelque chose.

Comment lire ? De haut en bas la première fois. Ensuite, utilisez le sommaire et surtout la partie 11 (« Je veux modifier X → je vais où ? »).

Sommaire

01	La grande image — le restaurant	
02	La carte des dossiers	
03	La base de données — le grand classeur	
04	Le serveur (API) — la cuisine	
05	L'application (Desktop) — la salle	
06	La sécurité — badges, vigiles et permissions	
07	Le trajet d'un clic — exemple concret	
08	Les Assistants IA	
09	La mise en production — comment une modification arrive en ligne	
10	L'environnement de travail local	
11	« Je veux modifier X → je vais où ? »	
12	Petit dictionnaire	
13	Les règles d'or avant de modifier	

1 La grande image — le restaurant

Imaginez que Fad Agency OS est un **restaurant**. Il a trois grandes parties :

PARTIE DU RESTAURANT	DANS LE LOGICIEL	DOSSIER	SON TRAVAIL
 La salle (ce que le client voit)	L'application à l'écran	<code>apps/desktop</code>	Afficher les pages, les menus, les boutons, les formulaires
 La cuisine (là où on prépare)	Le serveur, appelé « API »	<code>apps/api</code>	Recevoir les demandes, appliquer les règles, préparer les réponses
 La réserve (là où tout est stocké)	La base de données	Supabase (en ligne) / Docker (en local)	Garder toutes les informations : clients, factures, tâches...

Le principe fondamental : la salle ne va **jamais** directement dans la réserve. Quand vous cliquez sur un bouton, la salle **passe commande** à la cuisine, la cuisine va chercher dans la réserve, prépare le plat, et le renvoie à la salle qui vous l'affiche. Ce trajet prend une fraction de seconde.

Autour du restaurant, il y a aussi :

- **GitHub** : le coffre-fort où toutes les versions du code sont conservées (l'historique complet, rien ne se perd).
- **Railway** : l'hébergeur — c'est lui qui fait tourner la cuisine et la salle sur Internet, 24h/24.
- **OpenAI** : le « cerveau » externe que les Assistants IA appellent pour rédiger leurs réponses.

2 La carte des dossiers

Voici les dossiers qui comptent, à la racine du projet (`D:\fadile agency\fad-agency-os`):

```
fad-agency-os/
├─ apps/
│   ├─ api/          ← LA CUISINE (le serveur)
│   │   ├─ src/modules/ ← un sous-dossier par métier (clients, factures, équipe...)
│   │   ├─ src/common/  ← les outils partagés (sécurité, numérotation...)
│   │   ├─ prisma/      ← LA RÉSERVE : le plan de la base + son historique
│   │   └─ Dockerfile   ← la recette de fabrication du serveur (+ marqueurs "rev")
│   │
│   └─ desktop/       ← LA SALLE (l'application visible)
│       ├─ src/features/ ← un sous-dossier par section de l'écran
│       ├─ src/lib/      ← les outils partagés (menu, téléphone vers le serveur...)
│       ├─ src/i18n/     ← les textes en 3 langues (fr / en / ar)
│       ├─ src/app/      ← le plan des pages (router)
│       └─ src/layout/   ← le cadre (barre latérale, barre du haut)
│
├─ docs/              ← tous les cahiers des charges et rapports (dont ce guide)
├─ railway.toml        ← les consignes pour l'hébergeur (version serveur)
└─ package.json        ← la carte d'identité du projet + les commandes
```

👉 **À retenir** : un « métier » (par exemple les factures) existe **en double** : une fois côté cuisine (`apps/api/src/modules/invoices`) et une fois côté salle (`apps/desktop/src/features/invoices`). C'est normal : l'un fait les calculs, l'autre fait l'affichage.

3 La base de données — le grand classeur

La base de données est un grand classeur de **95 tables** (une table = un type de fiche). Le plan complet se trouve dans **un seul fichier** : `apps/api/prisma/schema.prisma`. Les tables se regroupent en familles :

FAMILLE	CE QU'ELLE STOCKE	TABLES PRINCIPALES
 Utilisateurs & sécurité	Les comptes de l'équipe, les rôles, les droits, l'historique des connexions et des actions	User, Role, Permission, LoginSession, AuditLog
 Clients & CRM	Les clients et les prospects (pas encore clients) avec leur suivi commercial	Client, Lead, LeadContact
 Services & abonnements	Le catalogue de ce que l'agence vend, et ce que chaque client a acheté	Service, ClientService, Subscription
 Projets & tâches	Le travail à faire : projets clients, tâches de l'équipe	Project, Task, ProjectComment
 Facturation	Devis, factures, échéanciers, paiements, et la numérotation FAC-2026-0001	Quote, Invoice, InvoicePayment, NumberSequence
 Finance / Comptabilité	Toutes les entrées et sorties d'argent, les dettes et créances	FinancialTransaction, Debt, Expense, Payment
 Trésorerie	Les comptes où l'argent est rangé (banque, caisse) et chaque mouvement	FinancialAccount, Movement, Reconciliation
 Portefeuilles pub	Les budgets publicitaires déposés par les clients et leurs recharges	Wallet, AdWallet, TopUpRequest, AdSpend
 Marketing	Campagnes et résultats, contenus, bibliothèque, calendrier, validations	Campaign, MarketingContent, Approval
 Équipe (RH)	Fiches employés, départements, congés, documents, évaluations, objectifs	TeamMember, Department, LeaveRequest, ManagerEvaluation
 Rapports	Les rapports générés : sections, chiffres, versions PDF, liens de partage	Report, ReportSection, ReportVersion, ReportShare
 Assistants IA	Les agents, conversations, messages, actions proposées, audits et propositions IA	AiAgent, AiConversation, AiMessage, AiAudit, AiProposal
 Portail client	Les comptes séparés de VOS clients (différents des comptes de l'équipe)	ClientUser
 Divers	Notifications, fichiers joints, notes, réglages	Notification, File, Setting, ConfigItem

⚠ **Règle capitale** : on ne modifie **jamais** la structure de la base directement. On modifie le plan (`schema.prisma`), on crée une **migration** (un petit fichier d'instructions dans `apps/api/prisma/migrations/`), on la teste en local, puis on l'applique en production avec une commande spéciale (voir partie 9). Le dossier migrations contient l'historique complet des ~27 changements de structure depuis le début.

4 Le serveur (API) — la cuisine

Le serveur habite dans `apps/api`. Chaque « métier » a son **module** (souvent un seul fichier) dans `apps/api/src/modules/`. Un module = un chef spécialisé qui ne s'occupe que de son rayon.

Toutes les adresses du serveur commencent par `/api/v1`. Exemple : la liste des clients se demande à `/api/v1/clients`.

MODULE (LE CHEF)	DOSSIER/FICHER	CE QU'IL GÈRE
Connexion (auth)	<code>modules/auth</code>	Vérifie email + mot de passe, délivre le « badge » de connexion
Utilisateurs / Rôles / Permissions	<code>modules/users</code> , <code>roles</code> , <code>permissions</code>	Les comptes de l'équipe et leurs droits
Clients	<code>modules/clients</code>	Les fiches clients
Prospects (CRM)	<code>modules/leads</code>	Les contacts commerciaux et leur pipeline
Catalogue de services	<code>modules/services</code>	Les prestations vendues et leurs prix
Projets	<code>modules/projects</code>	Les projets clients
Tâches	<code>modules/tasks</code>	Les tâches de l'équipe (assignation, statuts, validation, étoiles)
Campagnes	<code>modules/campaigns</code>	Les campagnes publicitaires et leurs résultats
Facturation	<code>modules/invoices</code>	Devis + factures + paiements + PDF
Comptabilité / Finance	<code>modules/accounting</code>	Les écritures, entrées/sorties, vue d'ensemble financière
Dettes & créances	<code>modules/debts</code>	L'argent dû (par l'agence ou par les clients)
Trésorerie	<code>modules/treasury</code>	Comptes, mouvements, transferts, rapprochement bancaire
Dépenses / Abonnements	<code>modules/expenses</code> , <code>subscriptions</code>	Frais de fonctionnement et charges récurrentes
Paiements / Portefeuilles	<code>modules/payments</code> , <code>wallets</code> , <code>wallet-transactions</code> , <code>ad-spend</code>	Encaissements et budgets pub
Marketing (6 chefs)	<code>modules/marketing/*</code>	Contenus, calendrier, bibliothèque, approbations, wallets pub, vue d'ensemble
Équipe	<code>modules/team</code> , <code>departments</code> , <code>planning</code> , <code>performance</code> , <code>internal-requests</code>	RH : membres, départements, congés, performance, demandes

MODULE (LE CHEF)	DOSSIER/FICHER	CE QU'IL GÈRE
Rapports	<code>modules/report-hub</code> , <code>reports</code>	Le générateur de rapports + les rapports détaillés
Assistants IA	<code>modules/ai-agents/*</code>	Le chat IA, les audits IA, les propositions IA
Statistiques	<code>modules/statistics</code>	Les chiffres des tableaux de bord
Portail client	<code>modules/portal</code>	L'espace où VOS clients se connectent
Notifications / Paramètres / Journal	<code>modules/notifications</code> , <code>settings</code> , <code>audit-logs</code>	La cloche d'alertes, les réglages, le registre « qui a fait quoi »

Trois protections automatiques s'appliquent à *toutes* les demandes sans que chaque chef ait à y penser (voir partie 6) : le vigile de connexion, le vigile des permissions, et le greffier qui note tout dans le journal d'audit.

5 L'application (Desktop) — la salle

L'application habite dans `apps/desktop`. Trois fichiers/dossiers commandent tout ce que vous voyez :

a) Le menu de gauche → `src/lib/constants.ts`

C'est LA liste officielle du menu. Chaque entrée y est décrite : son nom, son icône, sa page, son groupe, et qui a le droit de la voir. Le menu actuel a 6 groupes : **Opérations, Marketing, Finance, Rapports, Assistants IA, Équipe**.

👉 Ajouter, enlever ou renommer une entrée du menu = modifier ce fichier (et parfois les traductions, voir c).

b) Le plan des pages → `src/app/router.tsx`

La « carte routière » : elle relie chaque adresse (ex. `/clients`) à la page qui doit s'afficher. Elle renvoie aussi les non-connectés vers la page de connexion.

c) Les textes en 3 langues → `src/i18n/ (fr.json, en.json, ar.json)`

Certains textes (notamment le menu) sont traduits ici. **Attention** : un test automatique vérifie que les 3 fichiers ont exactement les mêmes clés — si on ajoute un texte en français, il faut l'ajouter aussi en anglais et en arabe, sinon le contrôle échoue.

d) Les sections de l'écran → `src/features/`

Un dossier par section, contenant les pages et leur logique :

DOSSIER	PAGES QU'IL CONTIENT
<code>features/auth</code>	La page de connexion de l'équipe
<code>features/dashboard</code>	Le tableau de bord général (admins)
<code>features/leads</code> , <code>clients</code> , <code>services</code> , <code>projects</code> , <code>tasks</code>	Les pages du groupe Opérations
<code>features/marketing</code>	Tout le pôle Marketing (vue d'ensemble, campagnes, contenus, bibliothèque, approbations...)
<code>features/accounting</code> , <code>invoices</code> , <code>treasury</code>	Les pages du groupe Finance
<code>features/reports-hub</code> , <code>reports</code>	Le générateur de rapports + rapports détaillés
<code>features/ai-agents</code>	Les Assistants IA (tableau de bord + chat + audits + propositions)
<code>features/team</code>	Tout le pôle Équipe (membres, organisation, performance...)

<code>features/portal</code>	L'espace client (connexion séparée sur <code>/portal/login</code>)
<code>features/settings</code> , <code>notifications</code> , <code>profile</code> , <code>archives</code>	Réglages, cloche, profil, archives

e) Les outils partagés → `src/lib/`

- `api-client.ts` — le « téléphone » vers le serveur : chaque page passe par lui, il ajoute votre badge automatiquement (délai d'attente : 15 s en général, 2 min pour le chat IA).
- `permissions.ts` — décide quels boutons/pages sont visibles selon votre rôle.
- `formatters.ts` — met en forme les montants (125 000 DZD), pourcentages, dates.
⚠ Le taux USD → DZD par défaut (218,5) est écrit ici et dans `constants.ts`.
- `export-excel.ts` — fabrique les fichiers Excel téléchargeables.

 **Bon à savoir** : certaines pages retirées du menu récemment (Dépenses, Abonnements, Statistiques, Archives, Campagnes & Résultats, Planning, Performance, Demandes internes...) **existent toujours** — leurs données sont conservées et elles restent accessibles en tapant leur adresse directement. Seule l'entrée du menu a été enlevée.

6 La sécurité — badges, vigiles et permissions

Le système fonctionne comme un immeuble sécurisé :

1. **Le badge (jeton JWT).** Quand vous vous connectez, le serveur vous remet un badge numérique. L'application le range dans un coffre (le trousseau sécurisé de Windows/macOS pour l'app installée) et le présente à chaque demande.
2. **Le vigile d'entrée** (`common/guards/jwt-auth.guard.ts`) vérifie le badge de chaque demande. Pas de badge, ou badge périmé → refusé.
3. **Le vigile des permissions** (`common/guards/permissions.guard.ts`) vérifie ensuite que vous avez **le droit précis** de faire cette action (ex. `invoices.create` pour créer une facture). Le Propriétaire et l'Admin passent toujours.
4. **Le greffier** (`common/interceptors/audit.interceptor.ts`) note chaque modification dans le **Journal d'audit** : qui, quoi, quand. Les mots de passe sont masqués avant d'être notés.

Les rôles (les « casquettes ») sont : Propriétaire, Administrateur, Manager, Media Buyer, Créatif, Comptable, Observateur. La grille complète « quel rôle a quel droit » est définie dans `apps/api/prisma/seed.ts` — c'est **la source de vérité** des permissions.

Deux mondes séparés : le badge de l'équipe et le badge du **portail client** sont différents et incompatibles — un client connecté à son portail ne peut jamais ouvrir une porte réservée à l'équipe, et il ne voit que **son propre** dossier.

7 Le trajet d'un clic — exemple concret

Vous cliquez sur « **Créer une facture** ». Voici ce qui se passe, en 8 étapes et moins d'une seconde :

1. 🧑 Vous remplissez le formulaire et cliquez « Enregistrer »
(page : `apps/desktop/src/features/invoices/...`)
2. 📞 La page appelle le serveur via le « téléphone »
(`apps/desktop/src/lib/api-client.ts` → `POST /api/v1/invoices`)
3. 🧑 Le vigile d'entrée vérifie votre badge (êtes-vous connecté ?)
4. 🧑 Le vigile des permissions vérifie votre droit « `invoices.create` »
5. ✅ Le contrôleur de formulaire (Zod) vérifie que les champs sont corrects
6. 🧑 Le chef « Factures » (`apps/api/src/modules/invoices`) fait le travail :
 - demande un numéro officiel à la machine à numéroter → `FAC-2026-0042`
 - enregistre la facture dans la base de données (via Prisma)
7. 📝 Le greffier note l'action dans le Journal d'audit
8. 📺 La réponse revient à l'écran : la nouvelle facture s'affiche dans la liste

Si quelque chose se passe mal (champ manquant, droit insuffisant, panne), le « traducteur d'erreurs » transforme le problème en message clair à l'écran — et le détail technique est gardé dans les journaux du serveur.

8 Les Assistants IA

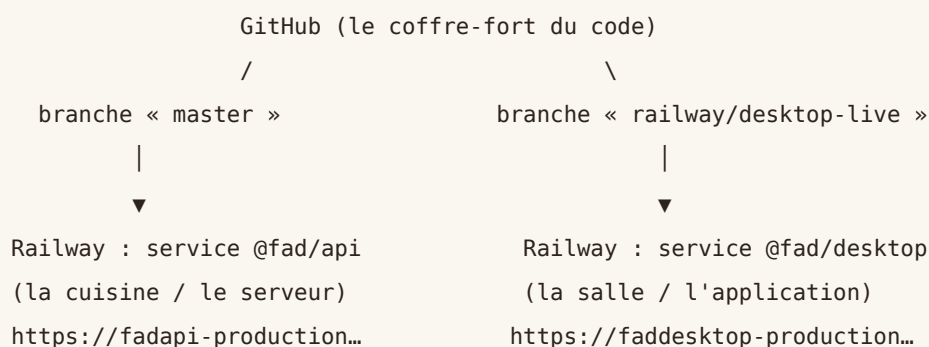
C'est le module le plus récent. Il combine **vos vraies données** + **un cerveau externe** :

Votre question → le serveur rassemble VOS chiffres (factures, tâches, campagnes...)
→ il les envoie à OpenAI (GPT-4o) avec des règles strictes
→ l'IA rédige la réponse en se basant UNIQUEMENT sur ces chiffres
→ la réponse s'affiche dans le chat

- **8 agents** spécialisés : Manager, Tâches & Projets, Marketing, Comptabilité, Rapports, Équipe, Audit, Proposition.
- **4 niveaux de prudence** : lecture seule → brouillon → action à confirmer → action sensible (approbation d'un responsable obligatoire). L'IA ne modifie jamais rien toute seule.
- **La clé** : le serveur a besoin de la variable `OPENAI_API_KEY` (réglée dans Railway, service @fad/api) et d'un compte OpenAI crédité. Sans clé, le chat fonctionne en « mode hors-ligne » : il affiche quand même vos chiffres, mais sans rédaction intelligente.
- **Où ?** Cuisine : `apps/api/src/modules/ai-agents/` (le fichier `llm.service.ts` est celui qui parle à OpenAI). Salle : `apps/desktop/src/features/ai-agents/`.
- Les agents **Audit** et **Proposition** produisent des PDF (rapport d'audit 15 sections, proposition commerciale avec 3 options de prix + devis) — ces parties fonctionnent **sans** clé IA.

9 La mise en production

Comment une modification faite sur l'ordinateur arrive-t-elle sur Internet ? Il y a **deux chemins parallèles**, un par service :



Les 3 règles à connaître :

1. **Le serveur (@fad/api)** ne se redéploie que si le fichier `apps/api/Dockerfile` change. C'est pour ça qu'on ajoute un commentaire `# revN: ...` en bas de ce fichier à chaque mise en production du serveur (on en est à rev30). C'est une bizarrerie de configuration Railway, documentée dans le fichier lui-même.
2. **L'application (@fad/desktop)** se redéploie à chaque envoi sur la branche `railway/desktop-live`. Cette branche est identique à master **sauf un fichier** : son `railway.toml` (les consignes pour l'hébergeur) qui pointe vers la recette desktop — il ne faut jamais l'écraser.
3. **La base de données de production** (Supabase) ne se met à jour **jamais automatiquement**. Quand la structure change, on lance à la main, depuis `apps/api` :

```
railway run --service "@fad/api" -- npx prisma migrate deploy
```

Railway prête les bons mots de passe le temps de la commande. (Cette manœuvre est volontaire : la base n'accepte que 15 connexions à la fois, et les mises à jour au démarrage bloquaient les déploiements.)

Après chaque mise en production de l'application : faire **Ctrl+Shift+R** dans le navigateur pour forcer le chargement de la nouvelle version (sinon l'ancienne reste en cache). Le pied de la barre latérale affiche la version exacte en ligne (numéro + code du commit + date) — pratique pour vérifier.

Le contrôle qualité : à chaque envoi sur GitHub, un robot (`.github/workflows/ci.yml`) vérifie que tout compile et que les ~320 tests passent. Il ne déploie rien, il surveille.

10 L'environnement de travail local

Pour travailler **sans toucher à la production**, tout tourne aussi sur l'ordinateur :

ÉLÉMENT	OÙ	COMMENT LE DÉMARRER
Base de données d'essai	Docker, port 5433	Ouvrir Docker Desktop, puis conteneur <code>fad-agency-postgres</code>
Le serveur	port 3001	<code>pnpm api:dev</code> (se recharge tout seul à chaque modification)
L'application	port 1420	<code>pnpm desktop:dev</code>

Compte d'essai local : `owner@fadagency.dz` / `ChangeMe123!` (⚠ valable **uniquement** sur la base locale — la production a son propre mot de passe).

Le circuit sûr d'une modification : modifier → tester en local → vérifier (compilation + tests) → envoyer sur GitHub → Railway déploie → vérifier en ligne. C'est le circuit suivi pour chaque module depuis le début.

11 « Je veux modifier X → je vais où ? »

Le tableau le plus utile de ce guide. 

JE VEUX...	J'OUVRE...
Ajouter/enlever/renommer une entrée du menu	<code>apps/desktop/src/lib/constants.ts</code> (+ <code>src/il8n/fr.json</code> , <code>en.json</code> , <code>ar.json</code> si le texte y est traduit)
Changer l' ordre des groupes du menu	<code>apps/desktop/src/layout/Sidebar.tsx</code> (la ligne <code>GROUP_ORDER</code>)
Modifier une page à l'écran (textes, boutons, colonnes)	<code>apps/desktop/src/features/<la-section>/...</code>
Changer un calcul ou une règle métier (ex. comment un total est calculé)	<code>apps/api/src/modules/<le-module>/...</code>
Ajouter un champ à une fiche (client, facture, tâche...)	1) <code>apps/api/prisma/schema.prisma</code> (le plan) → 2) créer une migration → 3) le module concerné (cuisine) → 4) la feature concernée (salle)
Changer qui a le droit de faire quoi	<code>apps/api/prisma/seed.ts</code> (la grille des rôles) + parfois une migration pour la production
Modifier les textes traduits	<code>apps/desktop/src/il8n/fr.json</code> + <code>en.json</code> + <code>ar.json</code> (les 3 ensemble !)
Changer le taux de change USD→DZD par défaut	<code>apps/desktop/src/lib/constants.ts</code> et <code>src/lib/formatters.ts</code>
Modifier le comportement des agents IA (leurs consignes, le modèle utilisé)	<code>apps/api/src/modules/ai-agents/llm.service.ts</code>
Changer la clé OpenAI ou une variable secrète	Railway → service @fad/api → onglet Variables (jamais dans le code !)
Modifier le format des numéros (FAC-2026-0001...)	<code>apps/api/src/common/numbering.ts</code>
Modifier un PDF généré (facture, rapport, audit)	Le module concerné côté cuisine (cherchez « PDFDocument » dans le fichier)
Voir qui a fait quoi dans le logiciel	Dans l'app : Journal d'audit — côté code : <code>apps/api/src/modules/audit-logs</code>
Comprendre un module métier en détail	<code>docs/spec-<module>.md</code> (les cahiers des charges : comptabilité, équipe, marketing, rapports, ai-agents)
Forcer un redéploiement du serveur	Ajouter une ligne <code># revN: description</code> en bas de <code>apps/api/Dockerfile</code> , puis envoyer sur master

12 Petit dictionnaire

MOT	EN CLAIR
API	La cuisine : le programme qui reçoit les demandes et manipule les données.
Frontend / Desktop	La salle : ce que vous voyez à l'écran.
Base de données	La réserve : le grand classeur où tout est rangé.
Module / Feature	Un chef spécialisé (côté cuisine) / une section de l'écran (côté salle).
Prisma	Le pont entre la cuisine et la réserve (il traduit le code en requêtes de base de données).
Migration	Un petit fichier d'instructions qui modifie la structure du classeur (nouvelle table, nouvelle colonne).
JWT / jeton / badge	Le badge numérique qui prouve que vous êtes connecté.
Permission	Un droit précis, ex. <code>invoices.create</code> = « peut créer une facture ».
Rôle	Une casquette (Propriétaire, Manager, Comptable...) qui regroupe des permissions.
Endpoint / route	Une « porte » du serveur, ex. <code>/api/v1/clients</code> .
Commit	Une photo du code à un instant T, enregistrée dans GitHub avec un message.
Branche	Une version parallèle du code (ici : <code>master</code> pour le serveur, <code>railway/desktop-live</code> pour l'app).
Déployer	Mettre en ligne la nouvelle version.
Railway	L'hébergeur qui fait tourner le serveur et l'application sur Internet.
Supabase	L'hébergeur de la base de données de production.
Docker	La boîte qui fait tourner la base de données d'essai sur votre ordinateur.
Seed	Le script qui remplit une base vide avec les rôles, permissions et le compte propriétaire.
CI	Le robot de contrôle qualité qui vérifie chaque envoi de code.
rev marker	Le commentaire <code># revN</code> en bas du Dockerfile qui force Railway à redéployer le serveur.

13 Les règles d'or avant de modifier

1. **Ne jamais modifier directement la production.** Toujours : local → test → GitHub → déploiement.
2. **Ne jamais écrire un mot de passe ou une clé dans le code.** Les secrets vivent dans les Variables Railway.
3. **Une modification de la base = une migration**, testée en local avant la production. Jamais de suppression de table sans vérifier ce qui l'utilise.
4. **Les 3 langues avancent ensemble** : tout texte ajouté dans `fr.json` doit exister dans `en.json` et `ar.json`.
5. **Ne pas toucher au `railway.toml` de la branche desktop-live** : c'est lui qui fait vivre l'application en ligne.
6. **Après un déploiement de l'app : Ctrl+Shift+R** pour voir la nouvelle version.
7. **Vérifier avant d'envoyer** : compilation (`tsc`), tests (`vitest`), construction (`build`). Si l'un des trois est rouge, on n'envoie pas.
8. **Le Journal d'audit est votre ami** : en cas de doute sur « qui a changé quoi », il a la réponse.
9. **Les cahiers des charges font foi** : chaque grand module a son `docs/spec-*.md` — c'est la référence de ce qui a été demandé et construit.
10. **Dans le doute, ne supprimez pas — archivez.** Le logiciel est construit sur ce principe (suppression douce partout).

Guide généré le 15/07/2026 à partir d'un inventaire complet du code (serveur, application, base de données, sécurité, déploiement). À mettre à jour quand l'architecture évolue.